

Helm Charts for Beginners (Kubernetes / OpenShift) — super simple + tutorial

1) What Helm is (plain language)

Think of **Helm** as an “app installer” for Kubernetes.

- Kubernetes runs your app using YAML files (Deployment, Service, etc.).
- A **Helm chart** is a **bundle** of those YAML files + a way to customize them.

So instead of managing 15 YAML files by hand, you say:

- “Install this chart”
- “Use these settings (values)”

2) Chart structure (what’s inside)

Here’s the typical folder structure:

Helm chart structure (what’s inside a chart folder)

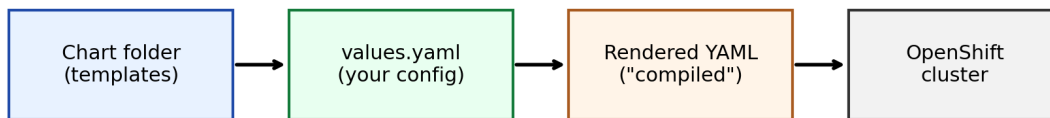
```
mychart/
  Chart.yaml      (name/version)
  values.yaml     (your settings)
  templates/      (YAML templates)
    deployment.yaml
    service.yaml
    route.yaml    (OpenShift)
  charts/         (optional deps)
```

In simple terms:

- `Chart.yaml`: chart name + version.
- `values.yaml`: your settings (image, replicas, ports...).
- `templates/`: YAML templates (they contain placeholders like `{{ .Values.replicaCount }}`).

3) The Helm workflow (the big picture)

Helm workflow: chart → rendered YAML → cluster



Helm does two jobs: 1) render templates to final YAML 2) apply that YAML to the cluster + store a release record

Your colleague said “compile a Helm chart” — Helm doesn’t compile like Java.

In Helm, “compile” usually means:

- **render templates → produce final YAML**

The command is:

```
``bash helm template myapp ./mychart -f my-values.yaml ``
```

That prints the exact YAML that would be applied to the cluster.

4) Hands-on tutorial: create your own Helm chart

We’ll create a tiny chart that deploys a container and exposes it.

Step 0 — Requirements

- Helm installed (`helm version`)
- Access to a Kubernetes/OpenShift cluster
- For OpenShift: `oc` installed (`oc version`) and login access

Step 1 — Create a starter chart

```
``bash mkdir -p helm-demo && cd helm-demo helm create hello-chart ``
```

This generates a chart with templates.

Step 2 — Open the important files

- `hello-chart/values.yaml` (your settings)
- `hello-chart/templates/deployment.yaml`
- `hello-chart/templates/service.yaml`

Step 3 — Configure the app you want to run

Edit `hello-chart/values.yaml` and set a real image. Example:

```
```yaml replicaCount: 1  
image: repository: nginx tag: "1.25"
service: type: ClusterIP port: 80 ```
```

### **Step 4 — Validate (lint)**

```
```bash helm lint ./hello-chart ```
```

Step 5 — Render (“compile”) to YAML locally (no cluster needed)

```
```bash helm template hello ./hello-chart ```
```

This is the best beginner trick:

- If the YAML output looks wrong, fix templates/values before installing.

---

## **5) Deploy to OpenShift (very simple)**

### **Step A — Login and choose a project**

```
```bash oc login https://api.:6443 oc new-project hello-project
```

or

```
oc project hello-project ```
```

Step B — Install the chart into that namespace

```
```bash helm install hello ./hello-chart -n hello-project ```
```

Check:

```
``bash helm list -n hello-project oc get all -n hello-project ``
```

### Step C — Upgrade (when you change values/templates)

```
``bash helm upgrade hello ./hello-chart -n hello-project ``
```

### Step D — Uninstall

```
``bash helm uninstall hello -n hello-project ``
```

---

## 6) OpenShift routing: Ingress vs Route (beginner view)

- Kubernetes often uses **Ingress**.
- OpenShift commonly uses **Route**.

So if your chart has `templates/ingress.yaml`, it might not be what your OpenShift cluster expects.

Two easy approaches:

- **Approach 1 (simple)**: keep using Service only (internal access) while learning.
- **Approach 2 (more OpenShift)**: add a Route template.

A very small Route template example (`hello-chart/templates/route.yaml`):

```
``yaml apiVersion: route.openshift.io/v1 kind: Route metadata: name: {{
include "hello-chart.fullname" . }} spec: to: kind: Service name: {{
include "hello-chart.fullname" . }} port: targetPort: http ``
```

(Charts usually make this optional with a value like `route.enabled: true`.)

---

## 7) Where does Helm store things in the cluster?

Important clarity:

- You usually **don't upload the chart** into the cluster as a "chart object".
- Helm:

1) reads chart files from your laptop/CI 2) renders YAML 3) applies YAML to the cluster 4) stores a **release record** in the namespace (often Secrets)

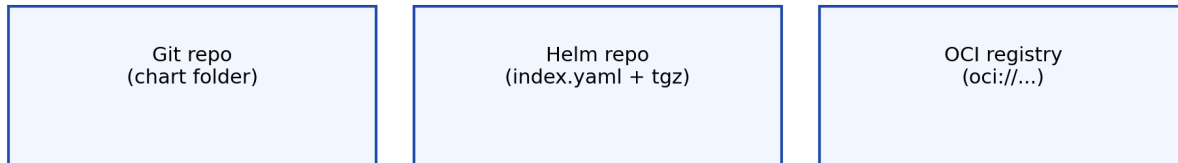
That's why you can do:

- `helm history`
- `helm rollback`

---

## 8) How to store/share a chart (real-world)

How teams store/share Helm charts (3 common ways)



CI/CD typically runs: `helm upgrade --install ...` (from Git, repo, or OCI)

### Option 1: Store the chart folder in Git (simplest)

Good for internal teams.

- Put `charts/myapp/` in your repo
- CI runs:

```
``bash helm upgrade --install myapp ./charts/myapp -n my-project -f
values-prod.yaml ``
```

### Option 2: Package it and publish to a Helm repo (classic)

Package:

```
```bash helm package ./hello-chart
```

creates something like `hello-chart-0.1.0.tgz`

```
```
```

A “Helm repo” is basically:

- the .tgz files
- an index.yaml

You can create the index:

```
``bash helm repo index . ``
```

Then host those files somewhere static (GitHub Pages, S3, Nexus/Artifactory).

### **Option 3: Push to an OCI registry (modern)**

Charts can live in an OCI registry (similar to images).

High-level flow:

```
``bash helm package ./hello-chart helm registry login <registry> helm
push hello-chart-0.1.0.tgz oci://<registry>/charts ``
```

Then install from the registry:

```
``bash helm install hello oci://<registry>/charts/hello-chart -n
hello-project ``
```

---

## **9) The simplest mental model**

- **Chart** = reusable app blueprint
- **Values** = your configuration
- **Render (“compile”)** = chart → final YAML
- **Install** = apply YAML to cluster + store release info

If you tell me which OpenShift setup you use (CI/CD tool, registry, GitHub/GitLab), I can add the exact recommended “best practice” flow for your environment.